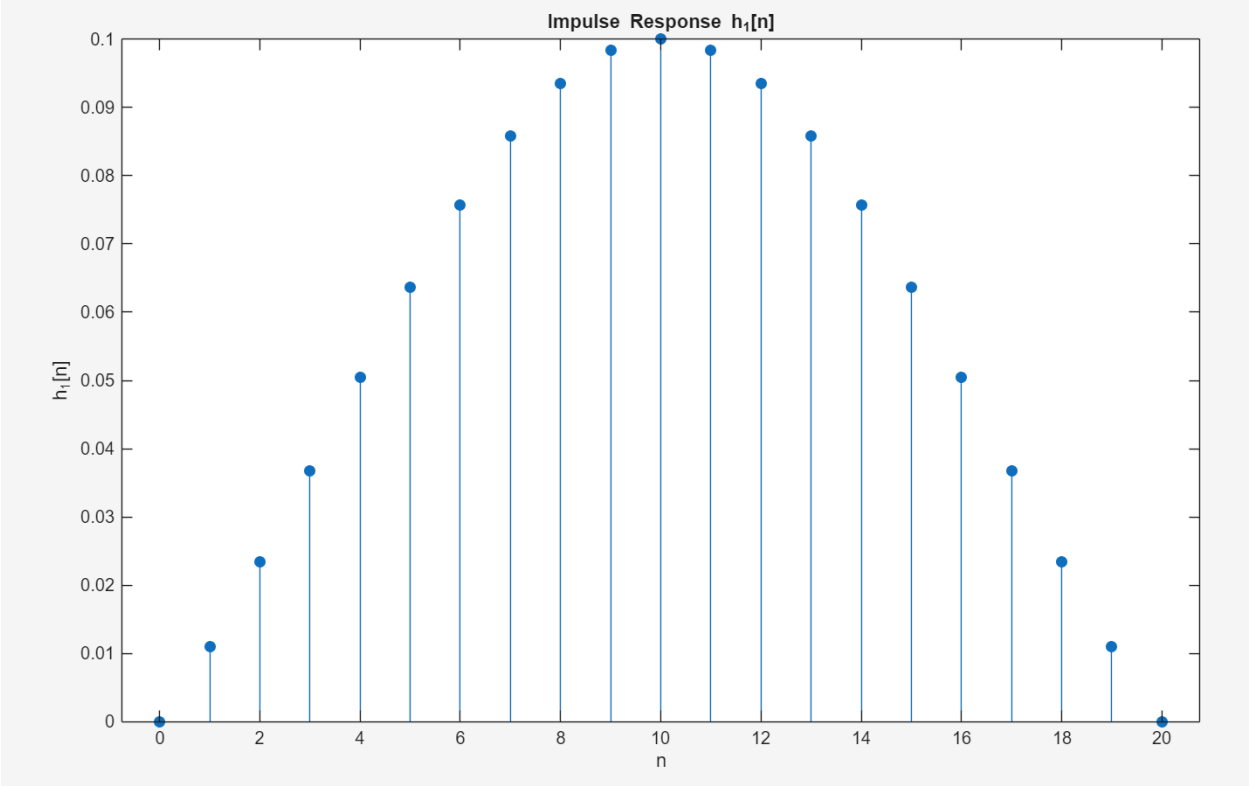
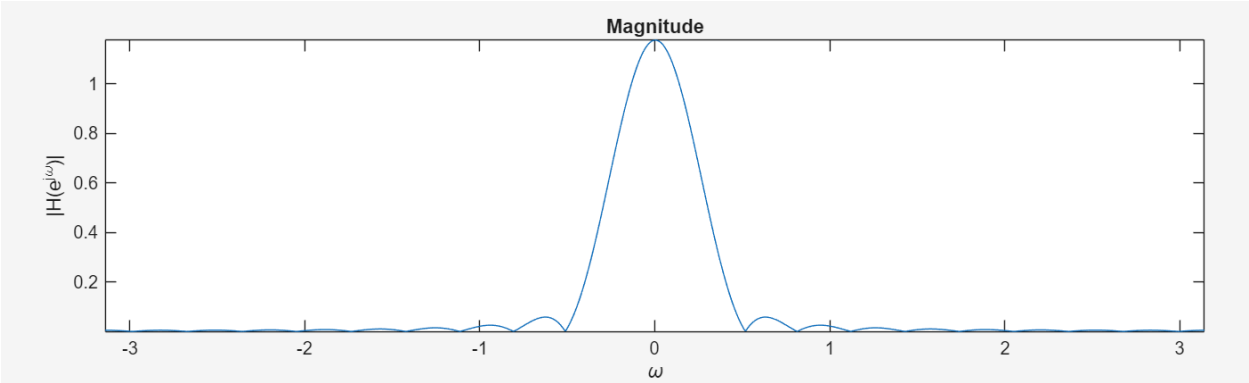
EE110B Lab#6

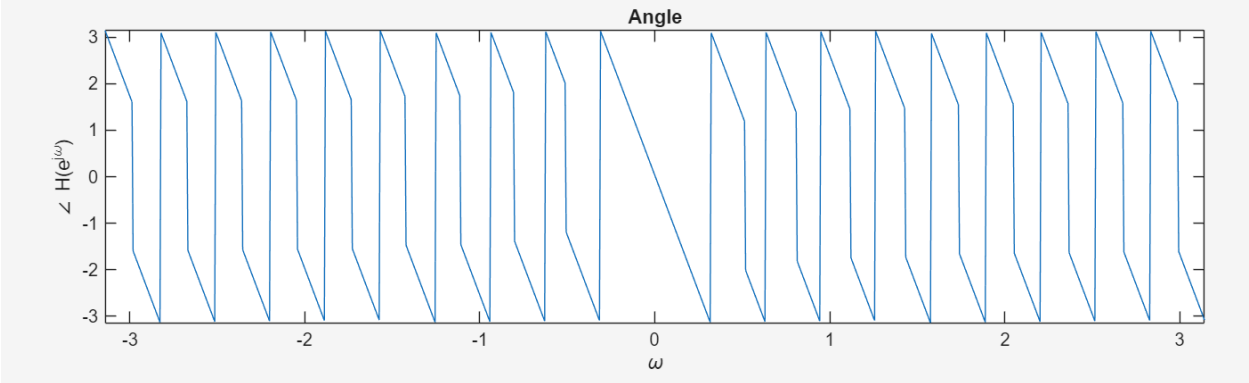
Nathan Taing

Understanding Low Pass Filters

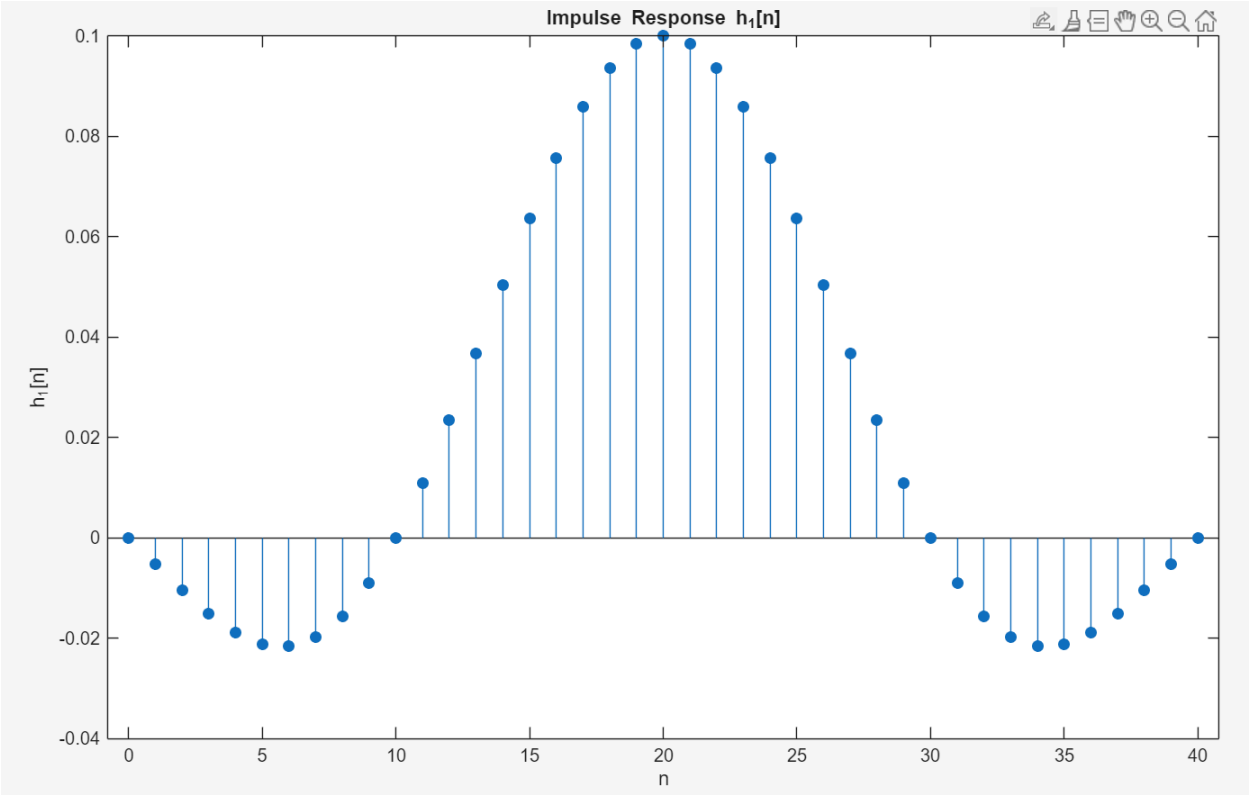
1) $n_0 = 10$

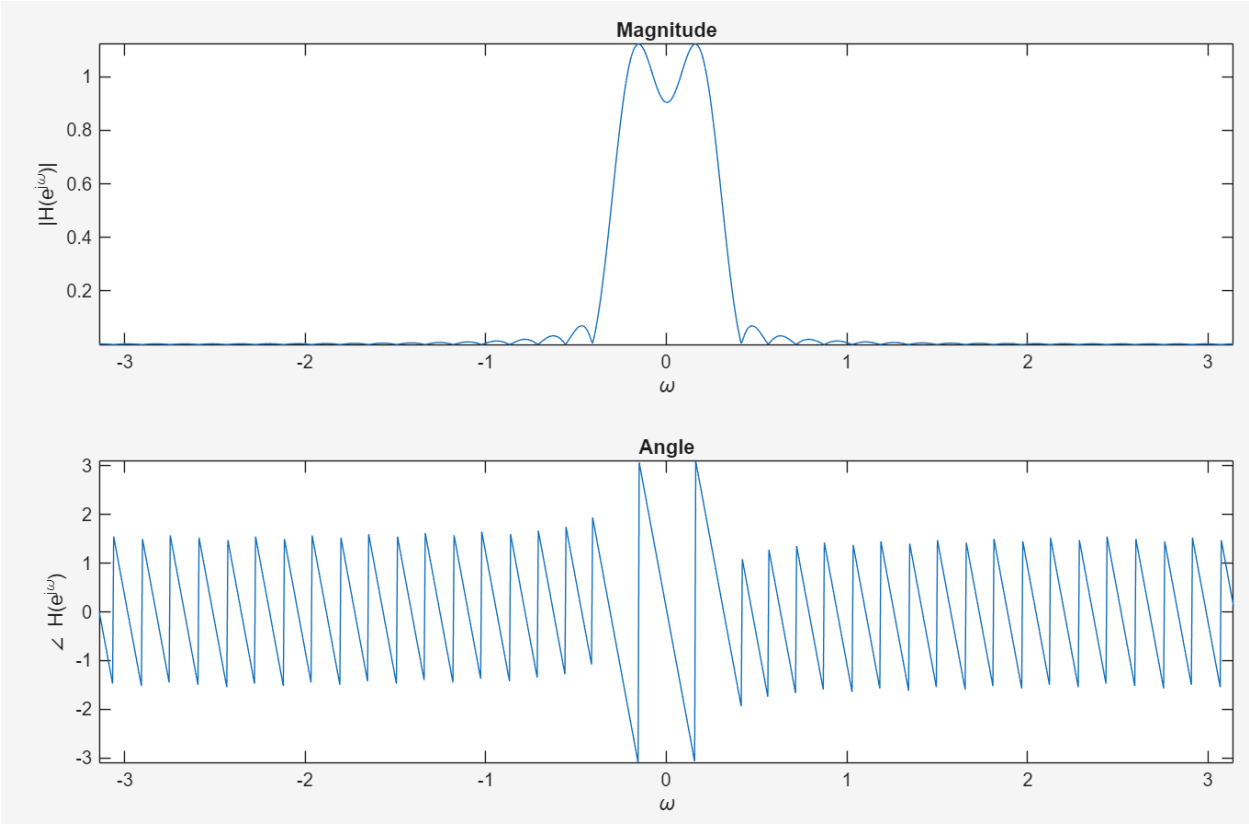




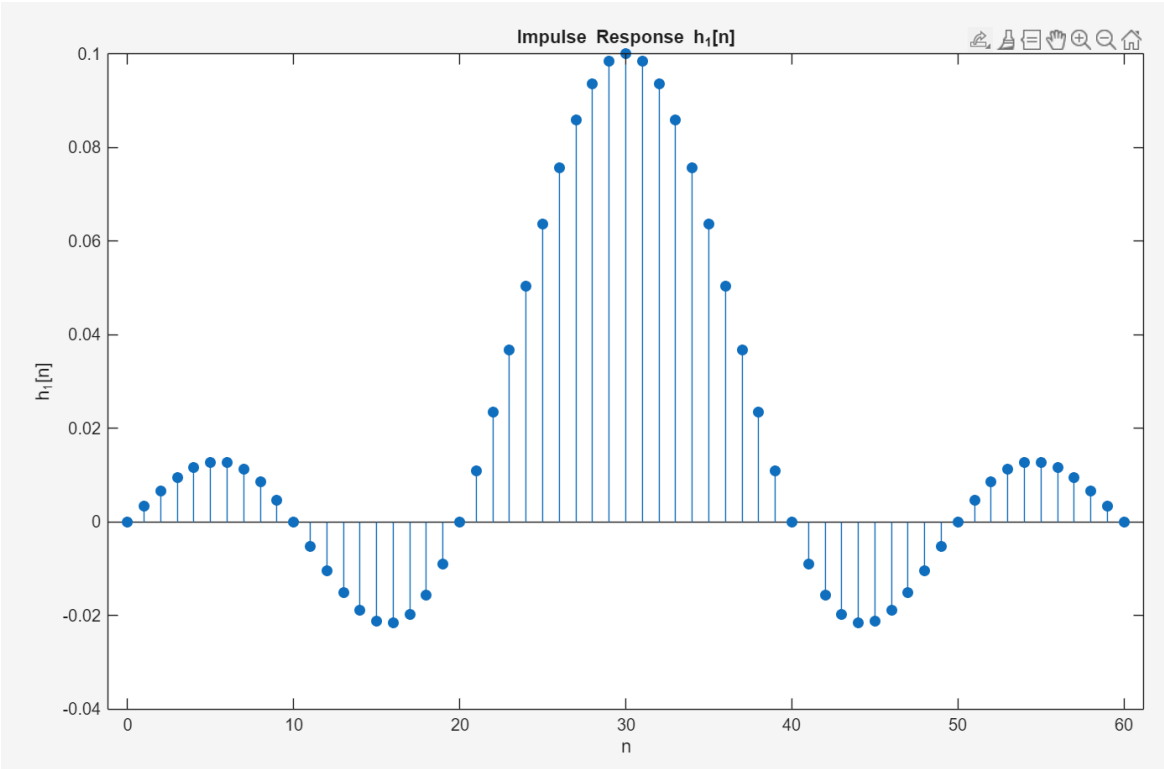


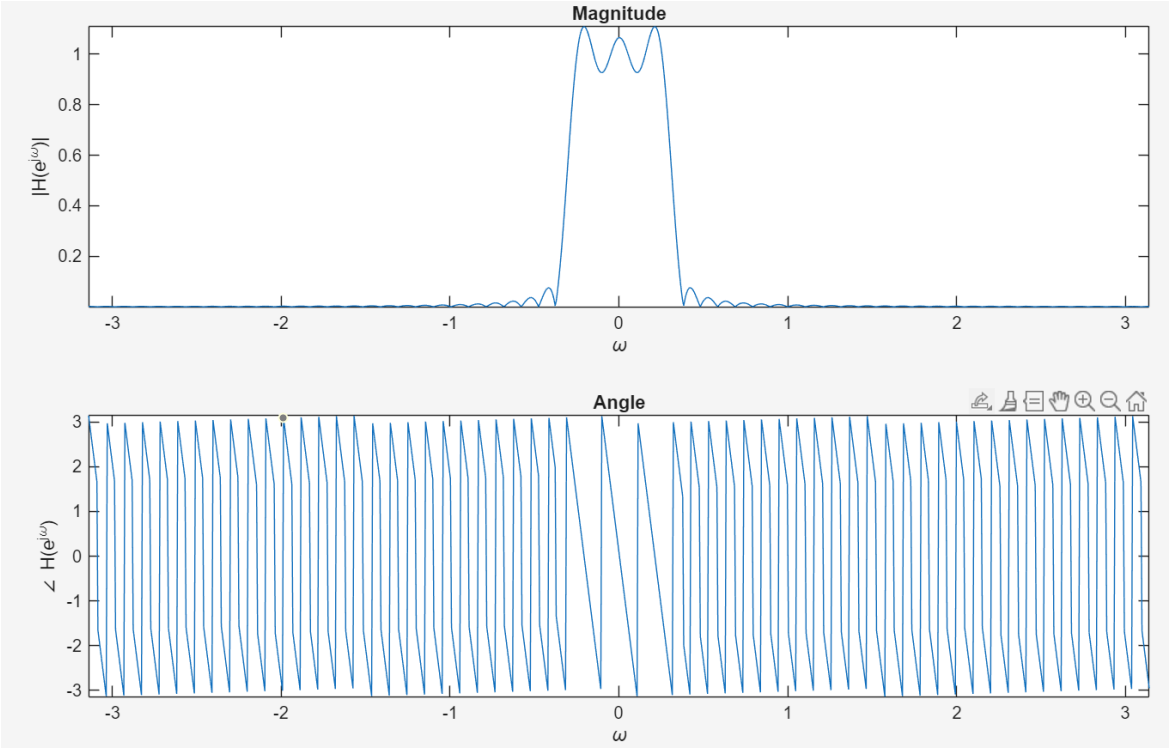
$n_0 = 20$


Impulse Response h1[n]
h1[n]
n
0.1
0.08
0.06
0.04
0.02
0
-0.02
-0.04
0
5
10
15
20
25
30
35
40


Magnitude
|H(e^jω)|
ω
1
0.8
0.6
0.4
0.2
-3
-2
-1
0
1
2
3
Angle
∠ H(e^jω)
ω
3
2
1
0
-1
-2
-3

$n_0 = 30$


Impulse Response h1[n]
h1[n]
n
0.1
0.08
0.06
0.04
0.02
0
-0.02
-0.04
0
10
20
30
40
50
60


Magnitude
|H(e^jω)|
ω
1
0.8
0.6
0.4
0.2
-3
-2
-1
0
1
2
3
Angle
∠ H(e^jω)
ω
3
2
1
0
-1
-2
-3

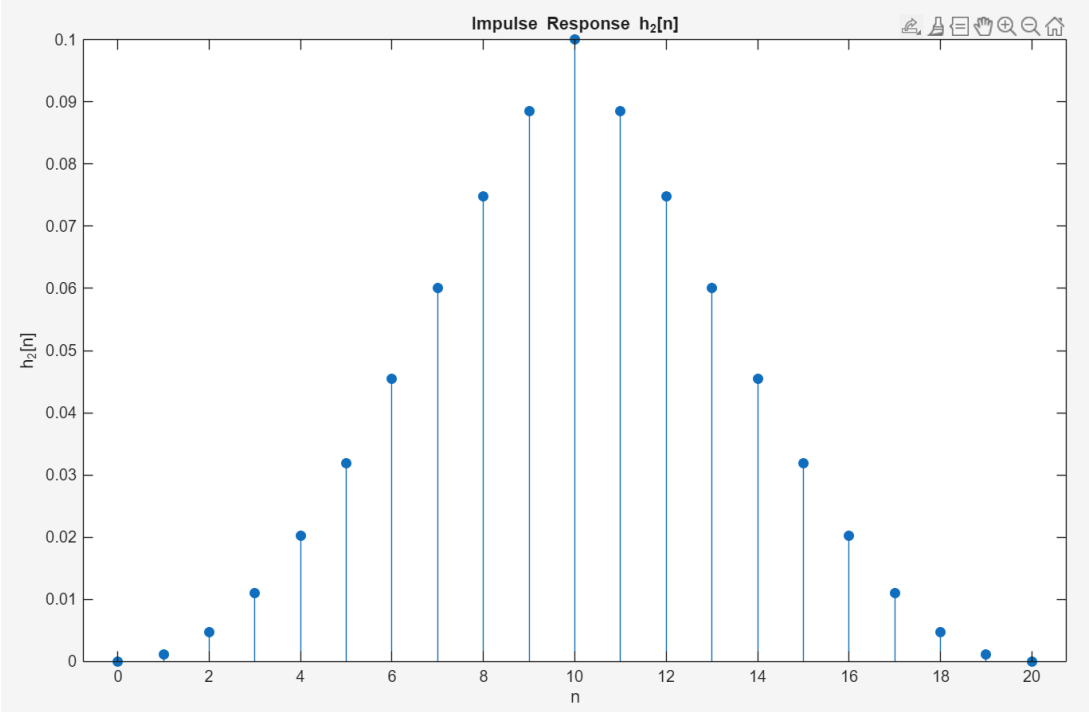
Code:

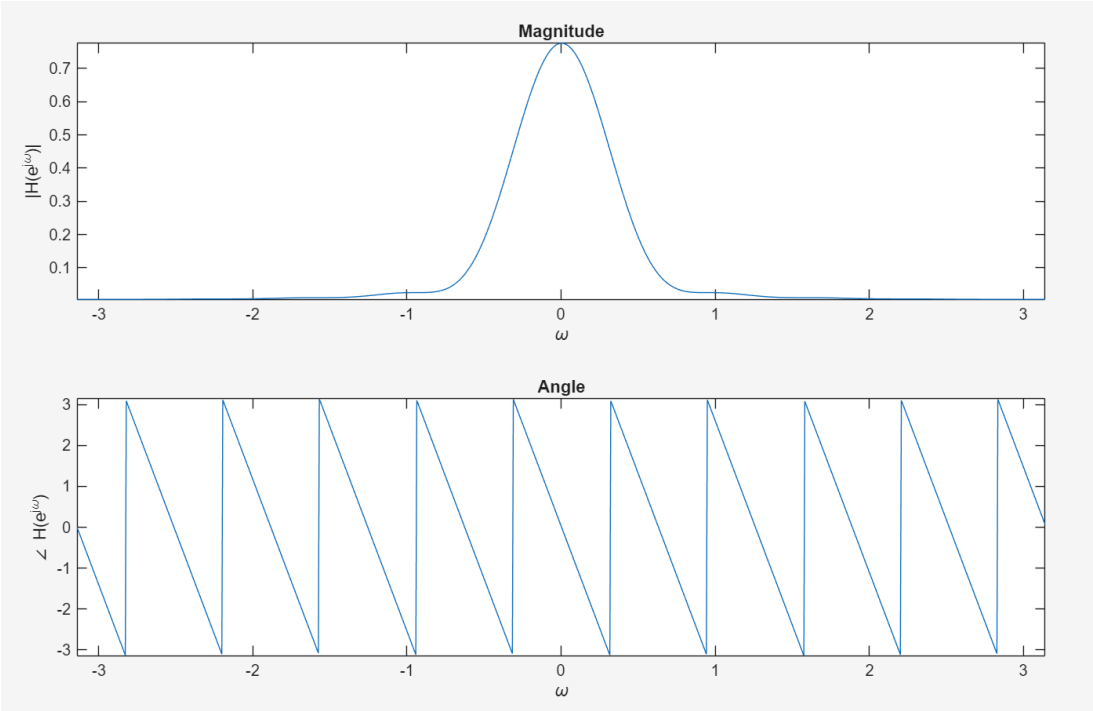
```
n0 = 30;
% Define n range after shifting
n = 0:2*n0;
m = n - n0;
% Create h1[n]
h1 = zeros(size(n));
h1(m == 0) = 1/10;
h1(m ~= 0) = sin(pi*m(m ~= 0)/10) ./ (pi*m(m ~= 0));
% Plot h1[n]
figure;
stem(n, h1, 'filled');
xlabel('n');
ylabel('h_1[n]');
title('Impulse Response h_1[n]');
%% Assume that windowed_resp is the time-domain response you calculated
windowed_resp = h1;
N_fft = 1024;     % Large enough number of samples to emulate
                  % the continuous DTFT.
% fftshift shifts the zero-frequency to center of the array
impulse_resp_fft = fftshift(fft(windowed_resp, N_fft));
% Discrete frequencies
freqs = linspace(-pi, pi, N_fft);
fig1 = figure;
subplot(2, 1, 1);
plot(freqs, abs(impulse_resp_fft));
xlabel('\omega')
ylabel('|H(e^{j\omega})|')
axis tight;
title('Magnitude')
subplot(2, 1, 2);
plot(freqs, angle(impulse_resp_fft))
xlabel('\omega')
ylabel('\angle H(e^{j\omega})')
axis tight;
title('Angle')
```

As n_0 increases, the impulse response becomes longer. We are keeping more samples of the ideal sinc response. Since the low-pass filter has an infinite impulse response, larger n_0 give a better

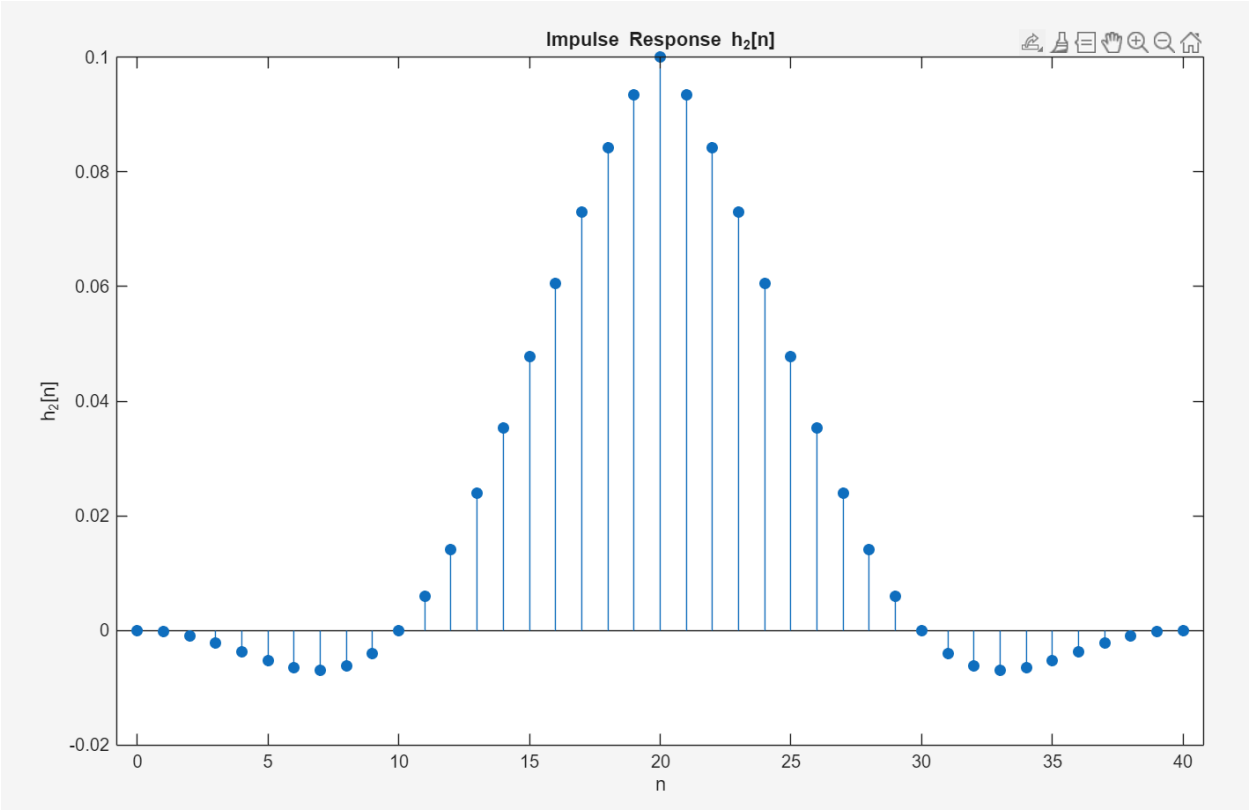
approximation of the original ideal filter. We can see the magnitude response become more rectangular as the max begins to level out and the magnitude spikes become more vertical. This can be seen by the phase of the signal increasing with each n_0. The magnitude and phase responses get closer to the ideal low-pass responses.

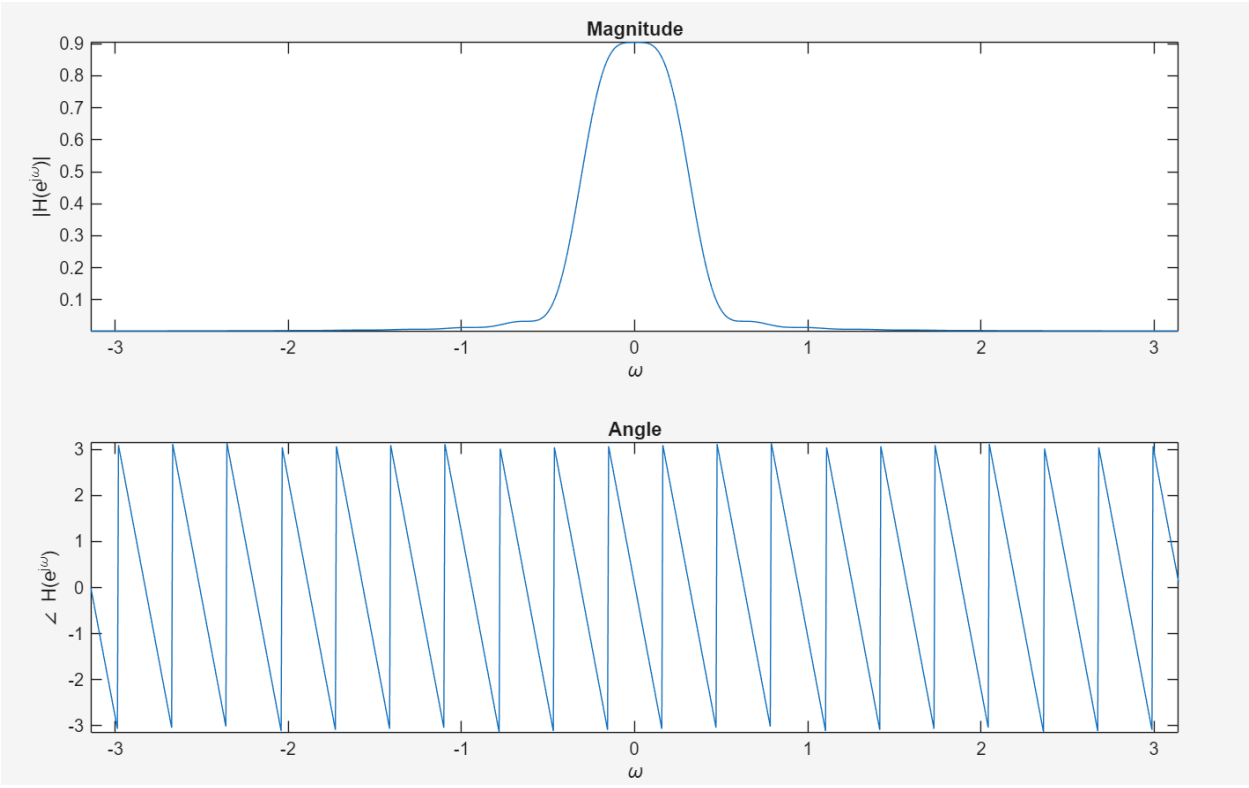
b)



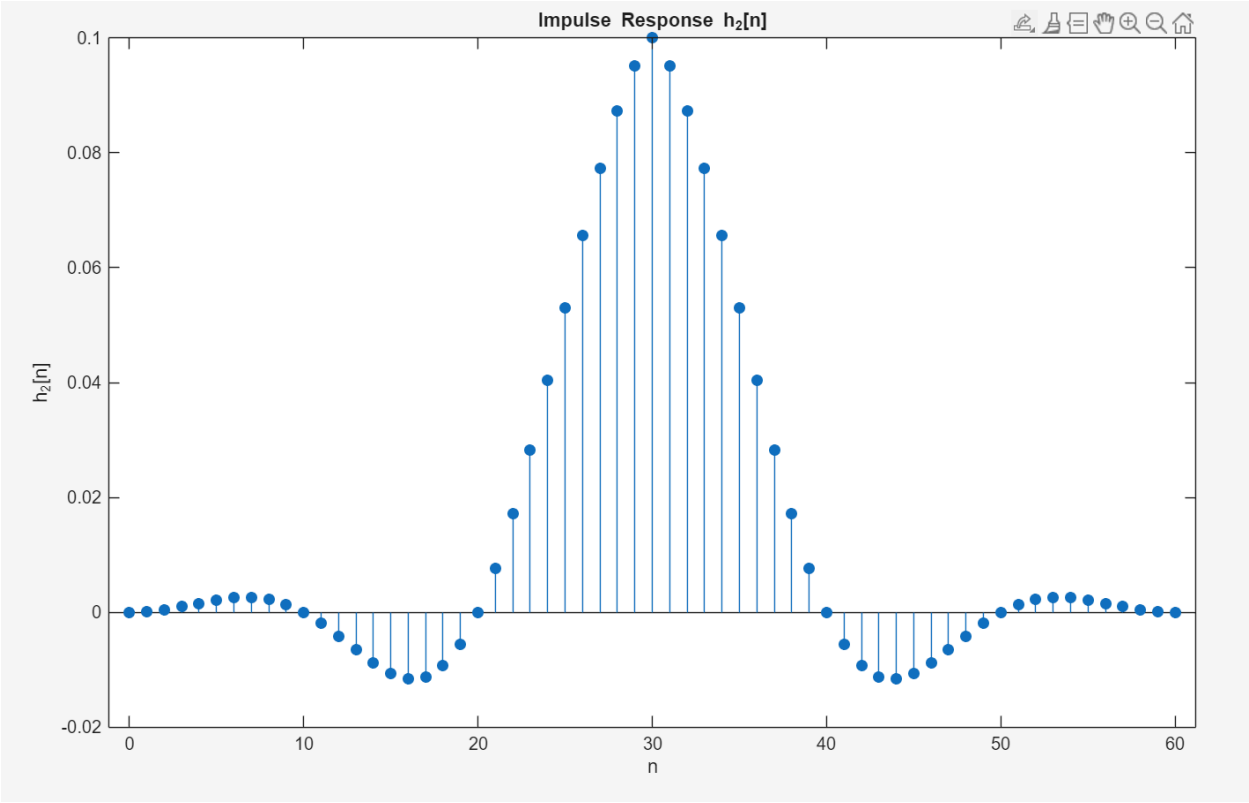


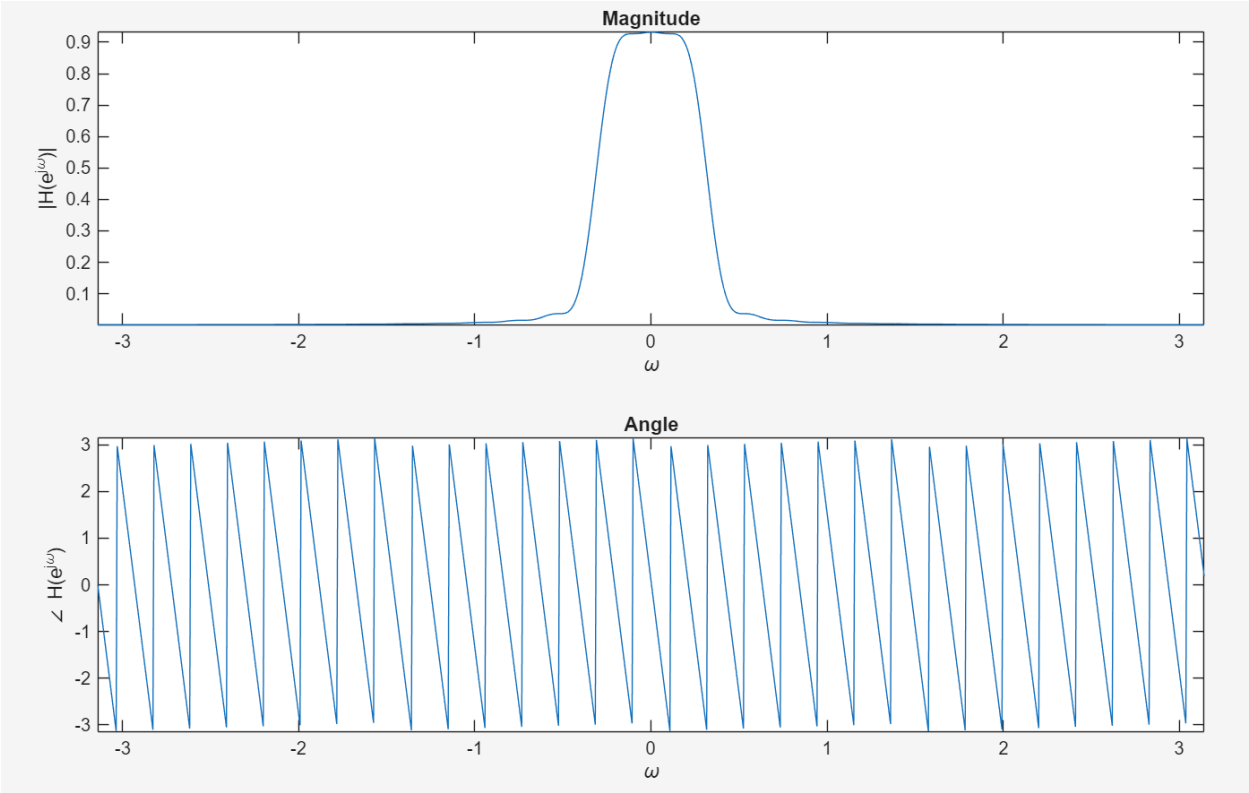
$n_0 = 20$


Impulse Response h2[n]
h2[n]
n
0.1
0.08
0.06
0.04
0.02
0
-0.02
0
5
10
15
20
25
30
35
40


Magnitude
|H(e^jω)|
ω
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
-3
-2
-1
0
1
2
3
Angle
∠ H(e^jω)
ω
3
2
1
0
-1
-2
-3

$n_0 = 30$


Impulse Response h2[n]
h2[n]
n
0.1
0.08
0.06
0.04
0.02
0
-0.02
0
10
20
30
40
50
60

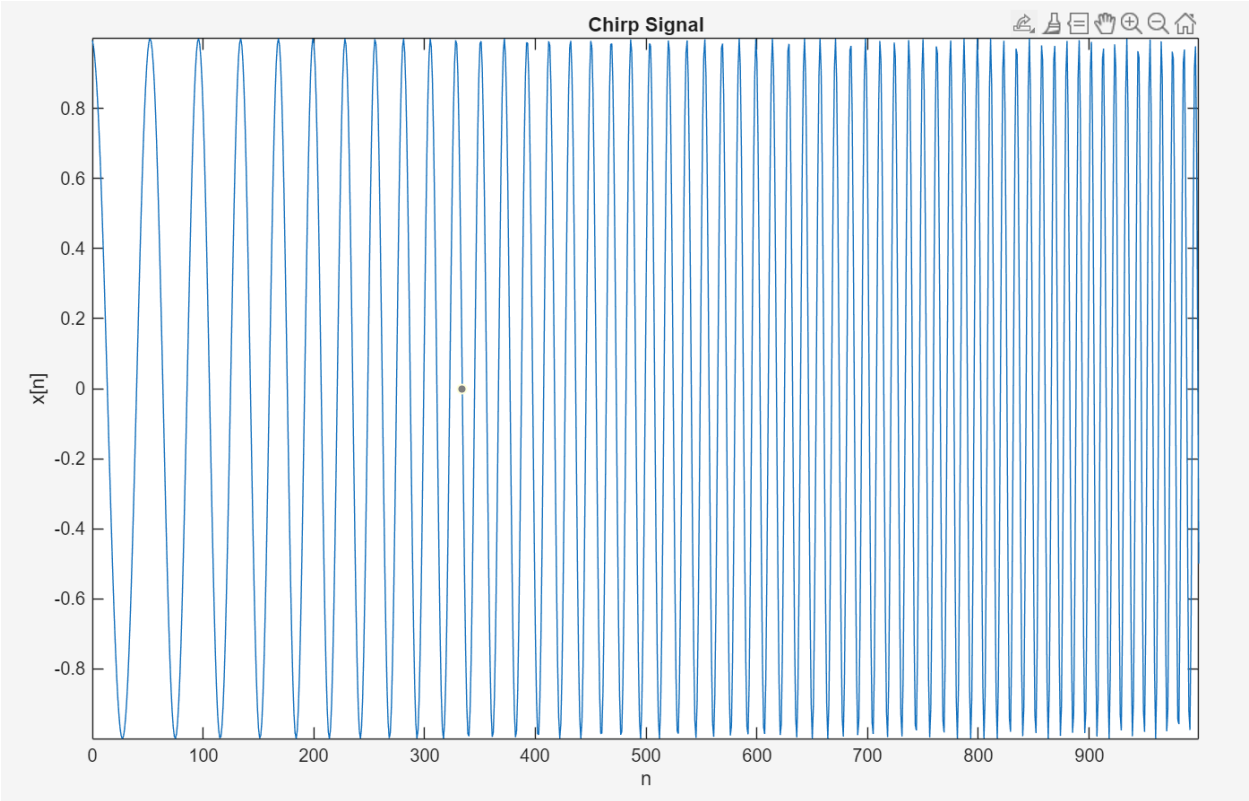

Magnitude
|H(e^jω)|
ω
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
-3
-2
-1
0
1
2
3
Angle
∠ H(e^jω)
ω
3
2
1
0
-1
-2
-3

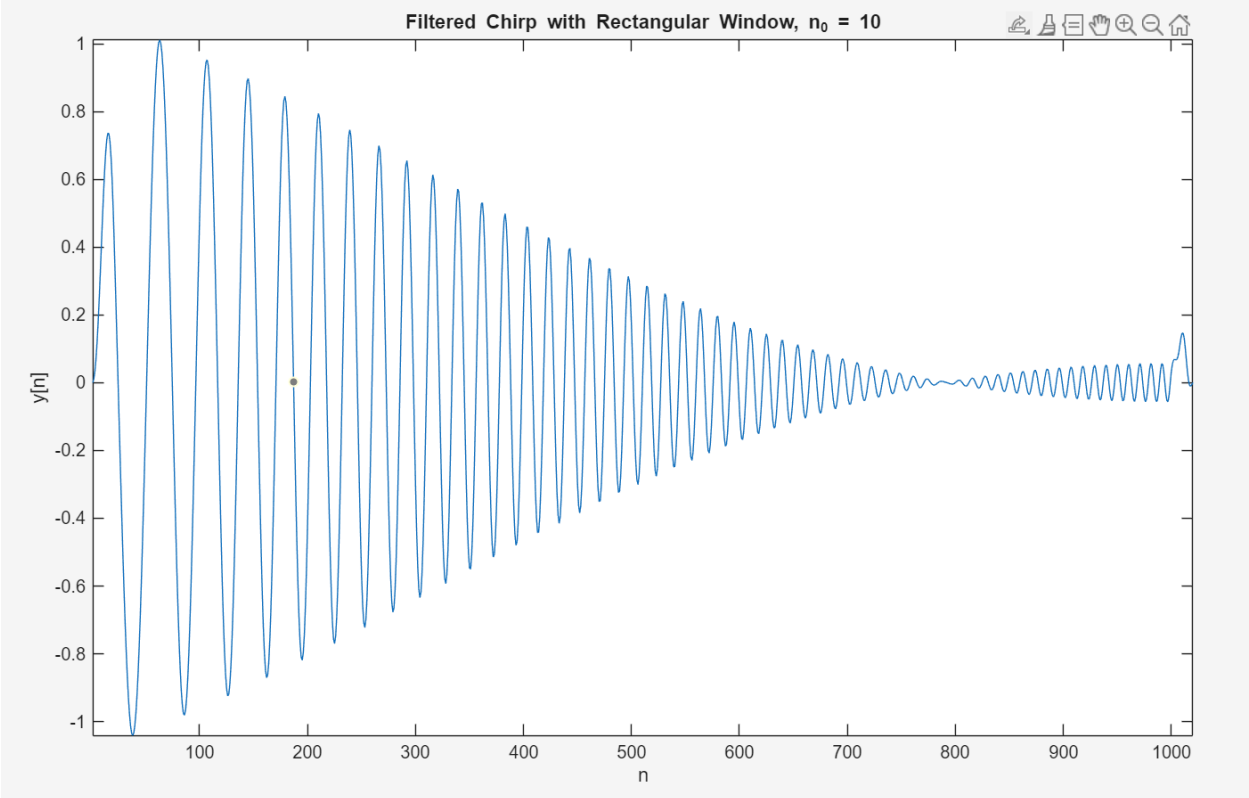
Code:

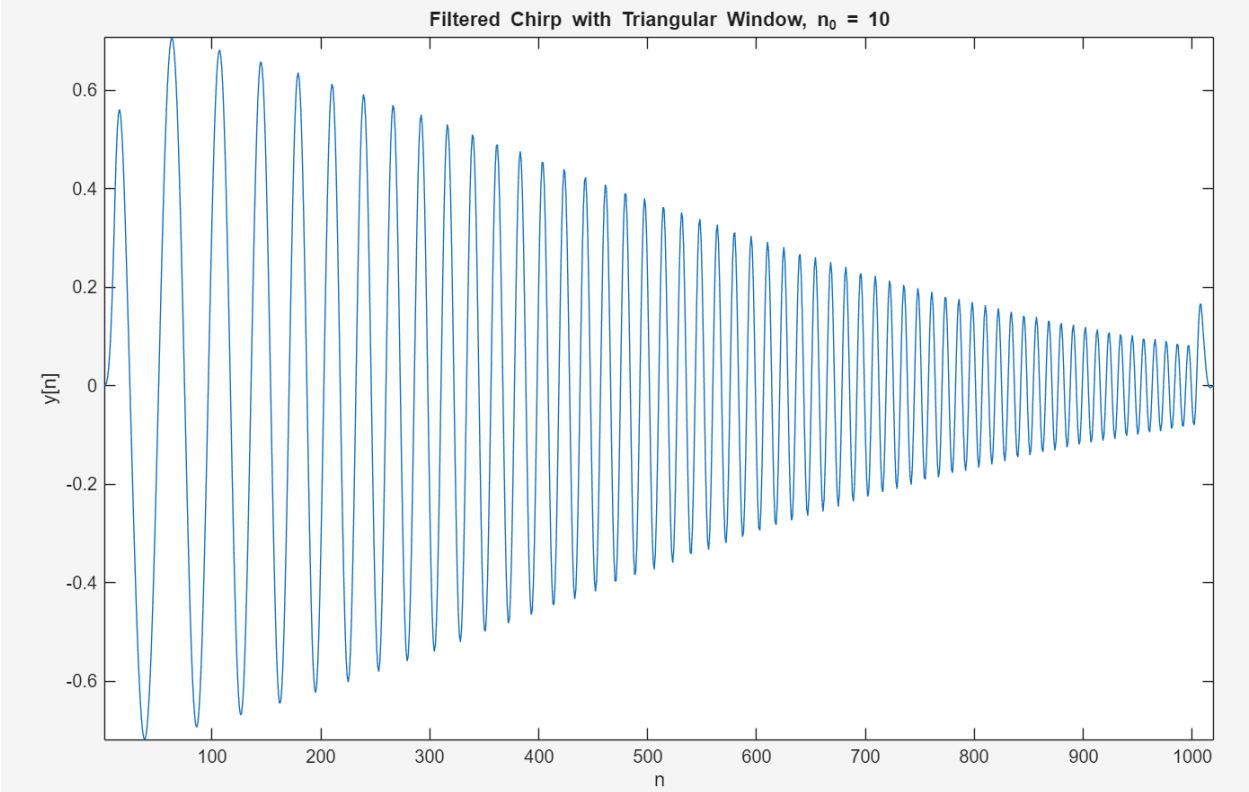
```
n0 = 30;
n = 0:2*n0;
m = n - n0;
h0 = zeros(size(n));
h0(m == 0) = 1/10;
h0(m ~= 0) = sin(pi*m(m ~= 0)/10) ./ (pi*m(m ~= 0));
t = 1 - abs(m)/n0;
h2 = h0 .* t;
figure;
stem(n, h2, 'filled');
xlabel('n');
ylabel('h_2[n]');
title('Impulse Response h_2[n]');
%% Assume that windowed_resp is the time-domain response you calculated
windowed_resp = h2;
N_fft = 1024;
impulse_resp_fft = fftshift(fft(windowed_resp, N_fft));
freqs = linspace(-pi, pi, N_fft);
fig1 = figure;
subplot(2, 1, 1);
plot(freqs, abs(impulse_resp_fft));
xlabel('\omega')
ylabel('|H(e^{j\omega})|')
axis tight;
title('Magnitude')
subplot(2, 1, 2);
plot(freqs, angle(impulse_resp_fft))
xlabel('\omega')
ylabel('\angle H(e^{j\omega})')
axis tight;
title('Angle')
```

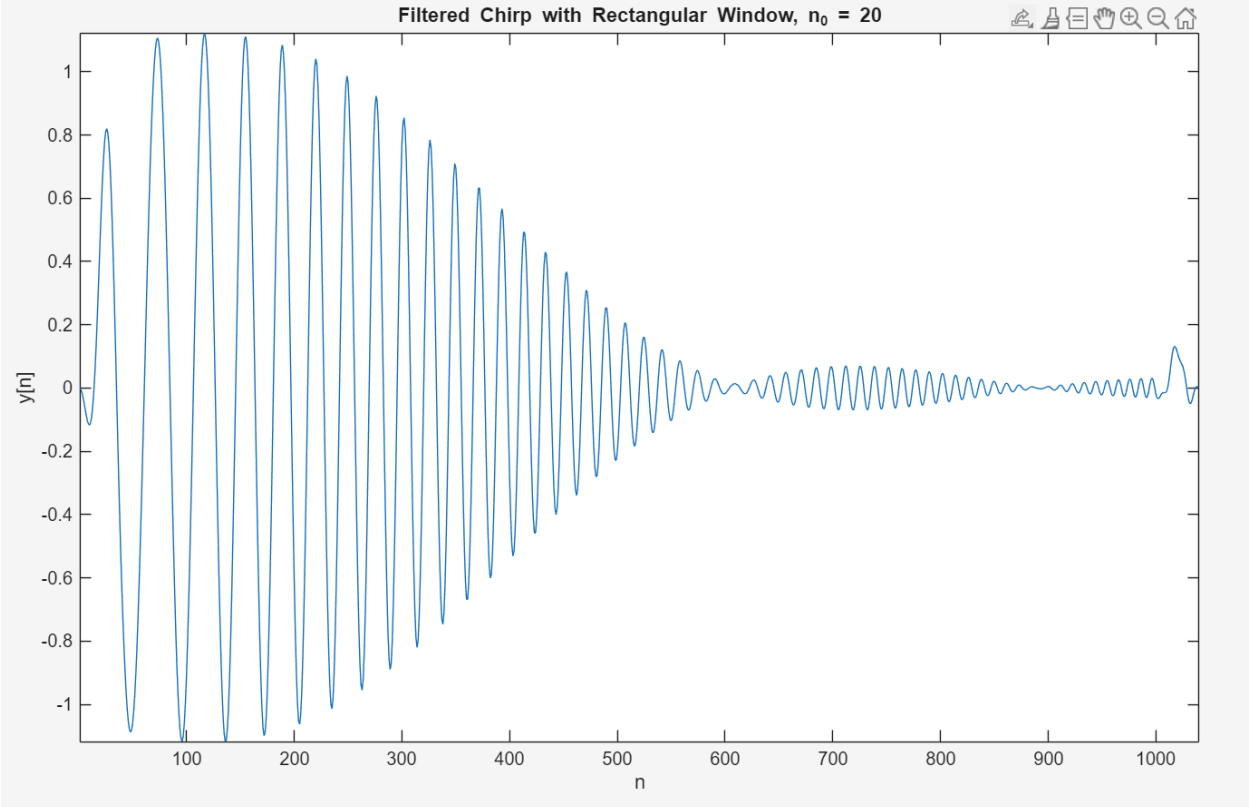
The triangular impulse responses appear to be smoother compared to the rectangular impulse response. This results in a more dramatic spike in the magnitude and phase responses that are recorded. There are also let ripples at the peak of the magnitude response. We can see the magnitude response appear more rectangular and the phase response is more consistent across all ns. This would lead me to say the triangular impulse response is better an approximating the $\left|H_0(e^{j\omega})\right|$. The response behavior is more in line with what an ideal low-pass filter should be.

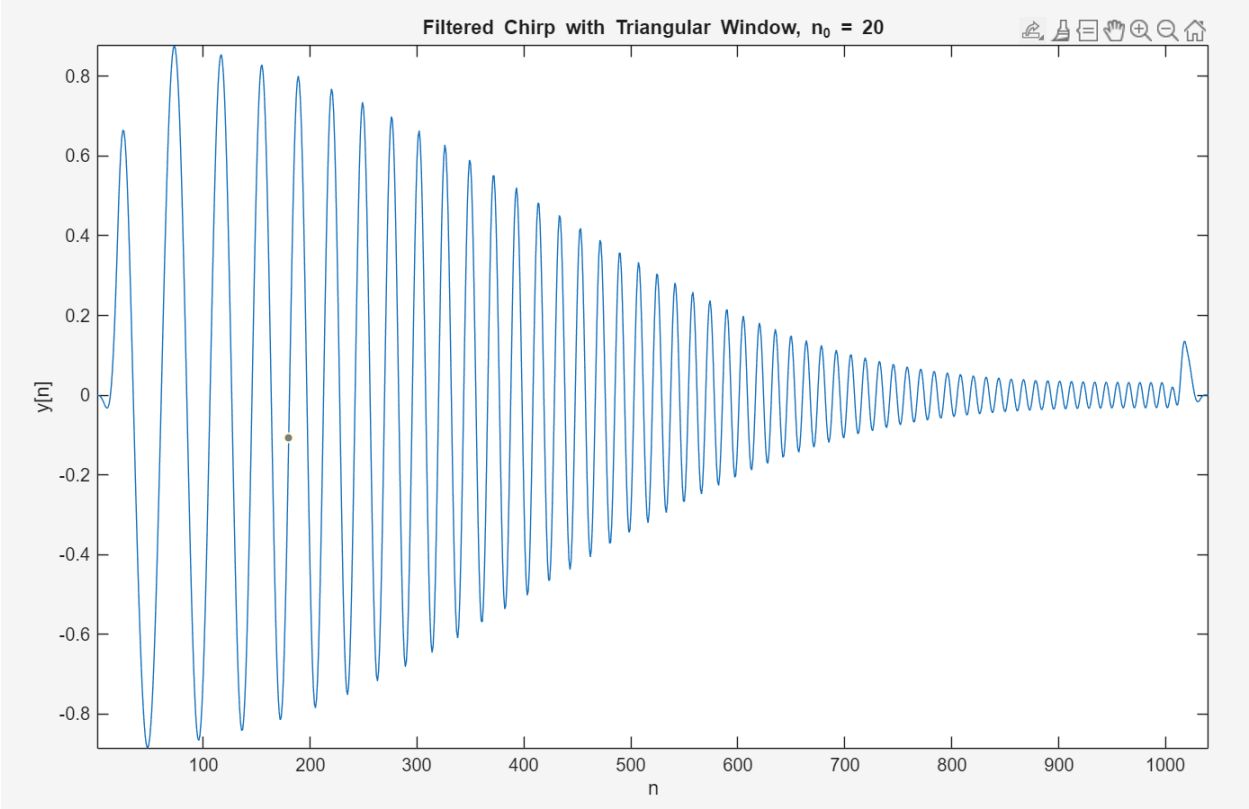
c)

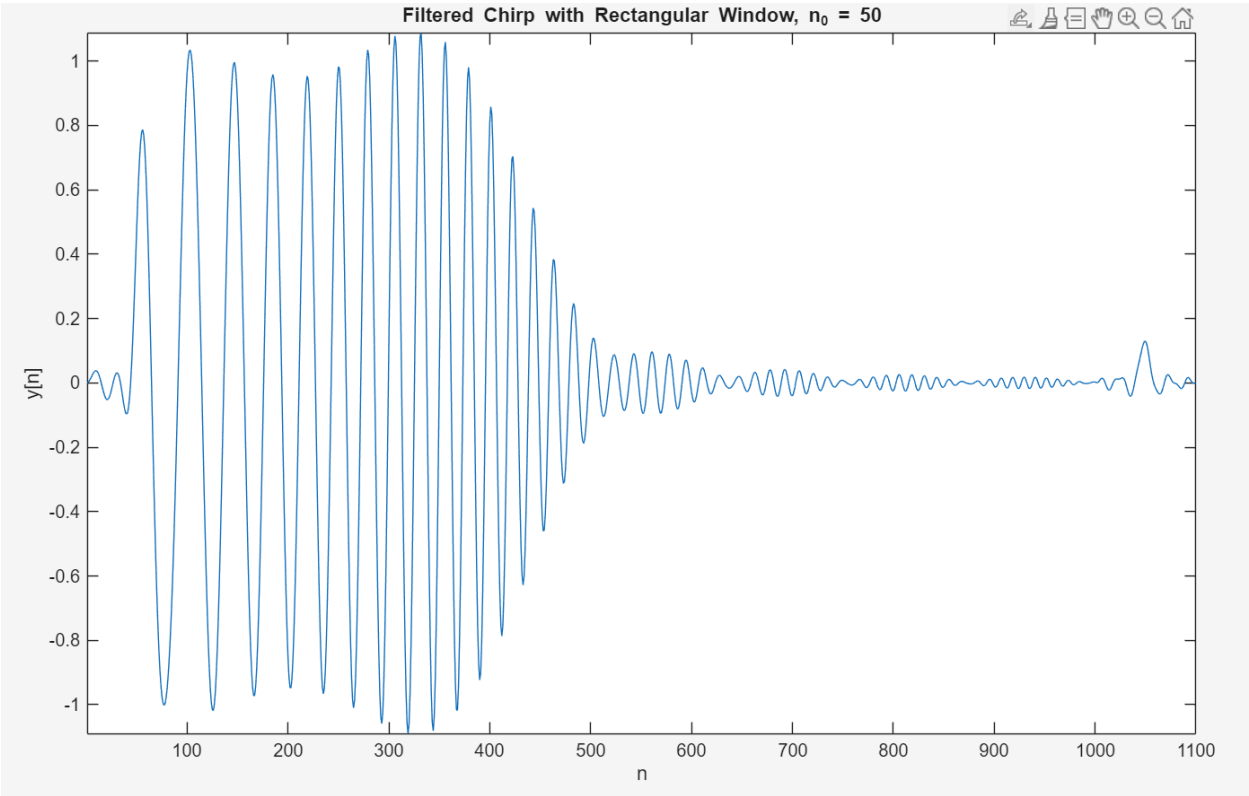

Chirp Signal
x[n]
n
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
0
100
200
300
400
500
600
700
800
900

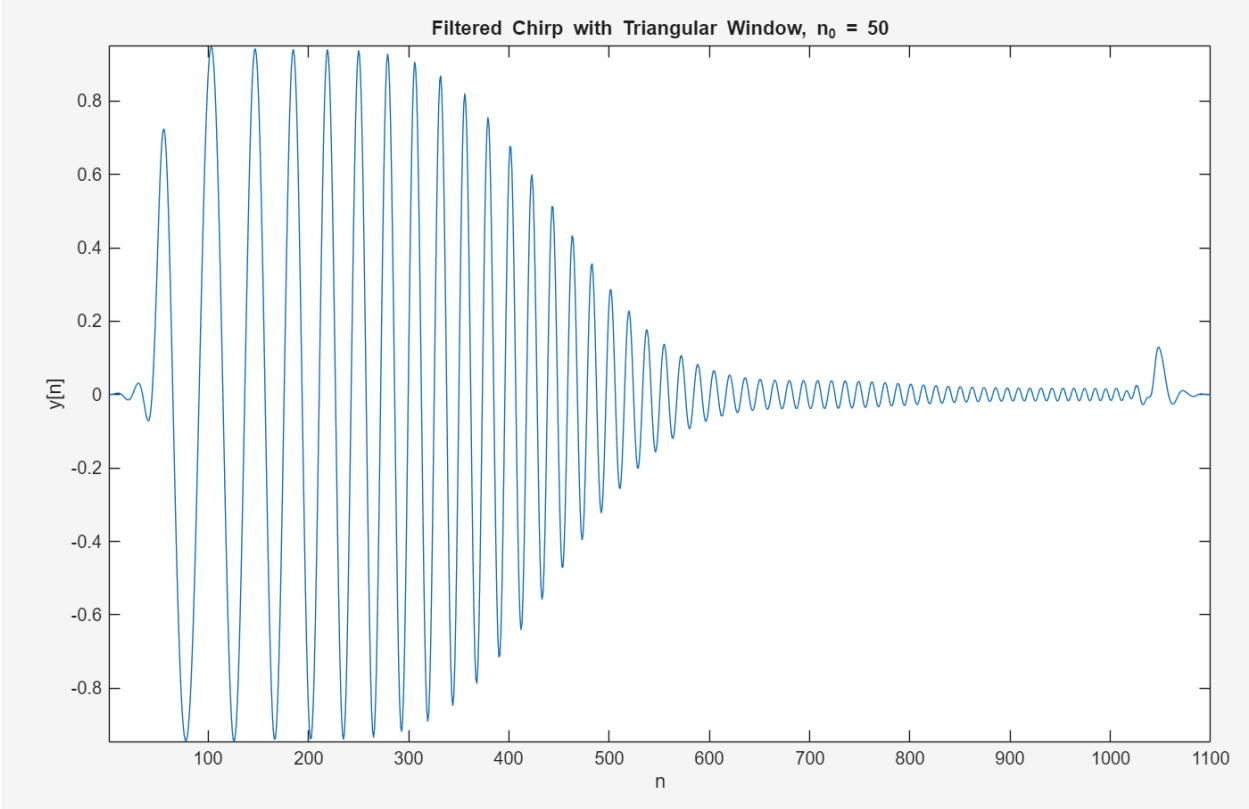

Filtered Chirp with Rectangular Window, n_0 = 10
y[n]
n
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-1
100
200
300
400
500
600
700
800
900
1000


Filtered Chirp with Triangular Window, n_0 = 10
y[n]
n
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
100
200
300
400
500
600
700
800
900
1000


Filtered Chirp with Rectangular Window, n_0 = 20
y[n]
n
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-1
100
200
300
400
500
600
700
800
900
1000


Filtered Chirp with Triangular Window, n_0 = 20
y[n]
n
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
100
200
300
400
500
600
700
800
900
1000


Filtered Chirp with Rectangular Window, n_0 = 50
y[n]
n
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-1
100
200
300
400
500
600
700
800
900
1000
1100



Code:

```
N = 1000;
n = 0:N-1;
w_init = pi/30;
w_final = pi/5;
% Create chirp signal
w = linspace(w_init, w_final, N);
phase = cumsum(w);
chirp_signal = cos(phase);
% Plot chirp signal
figure;
plot(n, chirp_signal);
xlabel('n');
ylabel('x[n]');
title('Chirp Signal');
axis tight;
n0_values = [10 20 50];
for i = 1:length(n0_values)
   n0 = n0_values(i);
   h_n = 0:2*n0;
```

```
    m = h_n - n0;
    % h0[n-n0]
    h0 = zeros(size(h_n));
    h0(m == 0) = 1/10;
    h0(m ~= 0) = sin(pi*m(m ~= 0)/10) ./ (pi*m(m ~= 0));
    % Rectangular window response h1
    h1 = h0;
    % Triangular window response h2
    t = 1 - abs(m)/n0;
    h2 = h0 .* t;
    % Convolve chirp with each filter
    y_rect = conv(chirp_signal, h1);
    y_tri = conv(chirp_signal, h2);
    figure;
    plot(y_rect);
    xlabel('n');
    ylabel('y[n]');
    title(['Filtered Chirp with Rectangular Window, n_0 = ', num2str(n0)]);
    axis tight;
    figure;
    plot(y_tri);
    xlabel('n');
    ylabel('y[n]');
    title(['Filtered Chirp with Triangular Window, n_0 = ', num2str(n0)]);
    axis tight;
end
```

The chirp signal begins with a low frequency and gradually increases to reach a high frequency by n = 999. We have been working with low-pass filters, which means the beginning of each of the signals will be clearer than the later portions of the chirp. This explains the triangular shape of each of the signals convolved with the chirp. The filter allows the low frequencies to pass through clearly (large oscillations of y[n]) and eventually the filter prevents the chirp from passing through (small oscillations of y[n]). The rectangular window creates a sharper cutoff, but a greater ripple. The triangular filter provides a smoother output that contains less ripple.